

The 6-Member Team Roster

The team is divided into 3 specialized tracks of 2.

- **AI & Graph Track:**

- **Member 1 (Agent Architect):** Responsible for LLM prompting, JSON extraction, system instructions, and RAG search logic.
- **Member 2 (Knowledge Graph Engineer):** Responsible for the Neo4j schema, database connection drivers, vector database embeddings, and query traversal.

- **Backend & Ingestion Track:**

- **Member 3 (Infrastructure Engineer):** Responsible for core server hosting, OAuth logins, real-time WebSockets, and third-party webhook connections.
- **Member 4 (Data Pipeline Engineer):** Responsible for the semantic data cleaner, the automated status state machine, and database updates.

- **Frontend UI/UX Track:**

- **Member 5 (Interactive Canvas Specialist):** Responsible for the interactive graph UI engine ([reactflow](#)), node positioning algorithms, and deep-linking layout.
- **Member 6 (Interface Developer):** Responsible for the general dashboard shell, navigation, workspace settings, profile views, and the chat components.

The 6-Week Timeline

Stage 1: The "Sandbox" Playground (zero-pressure learning)

Duration: Week 1 (Days 1–7)

Focus: Learning tools in complete isolation through miniature throwaway scripts before building the main app.

- **Member 1 (AI):** Builds a localized script that forces an LLM to respond only in structured, valid JSON parsing strings, ensuring no conversational fluff.
- **Member 2 (Graph):** Sets up a cloud Neo4j instance and uses the console to manually draw a 3-node mock ecosystem (**Developer** → **Wrote** → **Commit**).

[Instead of setting up a complex AI, they are just playing with a free database to practice drawing lines between three dots (nodes). They are also practicing writing prompts that force the AI to answer in clean code formatting (JSON) instead of talking like a human chatbot.]

- **Member 3 (Backend):** Configures the base Express/FastAPI server boilerplate and sets up **ngrok** to expose the local machine to the web.
Member 4 (Backend): Registers a test application with GitHub Developer settings to configure and test raw, unparsed webhook event collection.

[They are learning how to catch webhooks. They are setting up a tiny local server and using a tool called *ngrok* (which acts like a temporary bridge from the internet to their computer) just to see if an action on GitHub can send a text alert to their terminal.]

- **Member 5 (Frontend):** Sets up the React repository with Tailwind CSS and builds a single route testing basic **reactflow** box links using static code array objects.
Member 6 (Frontend): Builds the core CSS styling theme, layout shell, sidebar navigation navigation layout, and dashboard wrappers.

[They are setting up the basic React project layout. They are practicing using a tool called *reactflow* on a completely blank page just to see if they can make boxes appear and connect them on a screen.]

Stage 1 Resultant: Six independent, fully functional mini-scripts proving technical feasibility. You obtain verified, working database connections, an exposed API server catching dummy notifications, and a static visual graph running on a local web browser.

Stage 2: Ingestion Pipes & The AI Blueprint Engine (building real intake pipes)

Duration: Weeks 2 & 3 (Days 8–21)

Focus: Turning raw user text ideas into active flowcharts and setting up live event data pipelines.

- **Member 1 (AI):** Crafts system instructions that break an app idea down into distinct structural tasks based on tech-stack dependencies (e.g., Database setup before API routing).

Member 2 (Graph): Code scripts that read the AI's generated task lists and programmatically injects them into Neo4j as connected task nodes.

[One person writes the master instructions (prompts) so the AI can read a basic app idea and correctly spit out a list of required coding steps. The second person writes the code that automatically takes that AI list and saves it as points inside the graph database.]

- **Member 3 (Backend):** Expands the live servers to natively handle Discord messaging arrays and Figma layout change notification arrays.
- **Member 4 (Backend):** Builds the "Semantic Data Normalizer" to scrub incoming platform event arrays into clean, uniform timeline data blocks.

[They expand their simple webhook alerts to listen to real updates from Discord messages and Figma design edits. Then, they build a "Normalizer"—a script that takes all those messy notifications from different platforms and turns them into clean, matching text formats.]

- **Member 5 (Frontend):** Binds the visual graph rendering canvas directly to the backend database endpoints to replace static mock files.
- **Member 6 (Frontend):** Implements the "Connect Workspaces" view UI, integrating the authentication screens for a team's tools.

[They build the visible screens where users can click around to manage their team profiles. They also hook up the `reactFlow` layout to the backend, so the AI's generated task steps suddenly render as a gorgeous, interactive roadmap on the screen.]

Stage 2 Resultant: An automated software design blueprint system. You obtain a functional dashboard where typing a raw app idea instantly generates a living, dependency-locked engineering roadmap, while backend servers actively listen to live notifications across Discord, GitHub, and Figma.

Stage 3: Git-Driven Automation & Profile Skill Mapping (making it automatic)

Duration: Week 4 (Days 22–28)

Focus: Eradicating manual project management tracking by transforming code actions into live state switches.

- **Member 1 (AI):** Designs an ingestion pipeline that reviews a team member's public GitHub history to identify dominant coding traits.
- **Member 2 (Graph):** Integrates the user profile schema models to hold AI-mapped language attributes and links specific developers to corresponding architecture tasks.

[They build a smart profile scanner. The AI automatically scans a team member's public GitHub history, looks at what languages they code in, and automatically gives them fun technical skill badges on their profile.]

- **Member 3 (Backend):** Code-engineers the "State Machine" checking incoming Git branches or pull request actions to verify developer movement.
- **Member 4 (Backend):** Hooks up the WebSocket engine (Socket.IO) to broadcast real-time data changes immediately to all active sessions.

[They build a "State Machine." This is just code that listens to GitHub. If a developer pushes code that says "Fixes Task #12", the backend catches it and automatically tells the database to mark Task #12 as completed.]

- **Member 5 (Frontend):** Creates specialized real-time UI canvas listeners that visually modify task node styling (color changes) when server updates arrive.
- **Member 6 (Frontend):** Renders automated experience badges onto the team profiles and links interactive form updates to user cards.

[They learn how to use WebSockets (real-time communication pipes). This ensures that the second a developer pushes code on GitHub, the task card changes color on the web screen instantly without requiring a page refresh.]

Stage 3 Resultant: A zero-configuration task tracking automation layer. You obtain an interface that automatically changes task node states from "In Progress" to "Completed" on the screen the exact moment a developer submits a matching pull request, without requiring manual card adjustments.

Stage 4: (Graph-RAG) & Companion Standup Bot (building the brain and voice)

Duration: Week 5 (Days 29–35)

Focus: Injecting deep contextual conversational abilities and building the outer communication interface.

- **Member 1 (AI):** Configures vector embeddings parsing for conversational textual chunks using models like Llama-3 or OpenAI embeddings.
- **Member 2 (Graph):** Combines vector similarity indices with Neo4j relational paths to handle deep relational tracking questions.

[They combine vector search (finding matching text meanings) with graph maps. When a user asks the chatbot, *"Who changed this file?"*, the code looks up the file data, tracks down the specific developer node it's linked to, and pieces together a smart answer.]

- **Member 3 (Backend):** Assembles an interactive Discord bot application that acts as the messaging interface for active team members.
- **Member 4 (Backend):** Constructs localized background cron schedulers to cleanly summarize a developer's tracking activity into individual digestible updates.

[They build a custom Discord Bot. This bot runs a daily timer that looks at what everyone completed and pings each team member in Discord with easy interactive buttons to quickly confirm their status updates.]

- **Member 5 (Frontend):** Code-configures markdown parsing tools to support active clicking, making sure deep-links inside responses successfully redirect to external assets.
- **Member 6 (Frontend):** Integrates a sleek, sliding chat interface canvas to stream text query responses from Clover AI's engine.

[They build the slide-out chat window where you talk to Clover AI, making sure it displays code blocks neatly and highlights clickable external links.]

Stage 4 Resultant: A cross-platform conversational search experience. You obtain an AI assistant that accurately responds to queries regarding code updates and structural context with clickable links to specific files, reinforced by a low-friction Discord bot that handles automated standup checking directly inside chat channels.]

Stage 5: System Hardening & End-to-End Simulation (break, test, fix)

Duration: Week 6 (Days 36–42)

Focus: Merging track work, performance optimization, and running realistic workload stresses.

- **All Track Members (Collaborative Focus):** The group of 6 splits up into an isolated mock deployment team. For three days, you build a miniature application, run standard code pushes, use custom Discord spaces, and actively test for bugs or broken webhooks. Remaining days are reserved for code cleanup, reducing API latency, and optimizing database indices.

Stage 5 Resultant: A Production-Ready MVP. You obtain a unified, completely validated system that securely maps out app ideas, automatically matches skills, and tracks multi-platform software developments without administrative project tracking upkeep.